



EMAC Firmware Driver Quick Start Guide

Product Version 1.6.0

September 2017

© 1996-2017 Cadence Design Systems, Inc. All rights reserved.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 800.862.4522. All other trademarks are the property of their respective holders.

Restricted Permission: This document is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this document, or any portion of it, may result in civil and criminal penalties. Except as specified in this permission statement, this document may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This document contains the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only in accordance with, a written agreement between Cadence and its customer.

Unless otherwise agreed to by Cadence in writing, this statement grants Cadence customers permission to print one (1) hard copy of this document subject to the following conditions:

1. This document may not be modified in any way.
2. Any authorized copy of this document or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement.
3. The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration.

Disclaimer: INFORMATION IN THIS DOCUMENT IS SUBJECT TO CHANGE WITHOUT NOTICE AND DOES NOT REPRESENT A COMMITMENT ON THE PART OF CADENCE. EXCEPT AS MAY BE EXPLICITLY SET FORTH IN A WRITTEN AGREEMENT BETWEEN CADENCE AND ITS CUSTOMER, CADENCE DOES NOT MAKE, AND EXPRESSLY DISCLAIMS, ANY REPRESENTATIONS OR WARRANTIES AS TO THE COMPLETENESS, ACCURACY OR USEFULNESS OF THE INFORMATION CONTAINED IN THIS DOCUMENT. CADENCE DOES NOT WARRANT THAT USE OF SUCH INFORMATION WILL NOT INFRINGE ANY THIRD PARTY RIGHTS, AND CADENCE DISCLAIMS ALL IMPLIED WARRANTIES, INCLUDING MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. CADENCE DOES NOT ASSUME ANY LIABILITY FOR DAMAGES OR COSTS OF ANY KIND THAT MAY RESULT FROM USE OF SUCH INFORMATION. CADENCE CUSTOMER HAS COMPLETE CONTROL AND FINAL DECISION-MAKING AUTHORITY OVER ALL ASPECTS OF THE DEVELOPMENT, MANUFACTURE, SALE AND USE OF CUSTOMER'S PRODUCT, INCLUDING, BUT NOT LIMITED TO, ALL DECISIONS WITH REGARD TO DESIGN, PRODUCTION, TESTING, ASSEMBLY, QUALIFICATION, CERTIFICATION, INTEGRATION OF CADENCE PRODUCTS, INSTRUCTIONS FOR USE, LABELING AND DISTRIBUTION, AND CADENCE EXPRESSLY DISAVOWS ANY RESPONSIBILITY WITH REGARD TO ANY SUCH DECISIONS REGARDING CUSTOMER'S PRODUCT.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227- 14 and DFAR252.227-7013 et seq. or its successor.

1. Overview

This document describes the basics of how to get up and running with the Cadence IP Core Driver.

- Prepare the Cadence Platform Services (CPS)
- Prepare interrupt handling
- Build the core-driver
- Use the core-driver to initialise the Cadence IP
- Implement additional features using the core-driver

2. Details

Here's how to get started:

1. *Prepare the Cadence Platform Services:*

This is covered in detail in the Cadence Driver Porting Guide, which is available at [doc/porting/porting_guide.pdf](#). A sample implementation for bare-metal systems is available in [reference_code/tests/cps_emac_bm](#), and one for linux is supplied at [reference_code/lrx_mod/src/cps_v2_linux.c](#). You may be able to use these without modification.

CPS is a platform-specific code to connect a given Cadence driver to the rest of your system. Not all of the features of CPS are required for this driver. You only need to implement the functions for uncached access to memory:

```
CPS_UncachedRead16, CPS_UncachedRead32,  
CPS_UncachedWrite16 and CPS_UncachedWrite32
```

These functions will be used by the core driver to access the memory in a portable manner. Note that the core driver only uses 32-bit accesses, but the sample tests also use 16-bit accesses. `CPS_CacheFlush()` / `CPS_CacheInvalidate()` are not used by this driver.

After this step you should have an object file implementing the CPS for your target system which you can link with later.

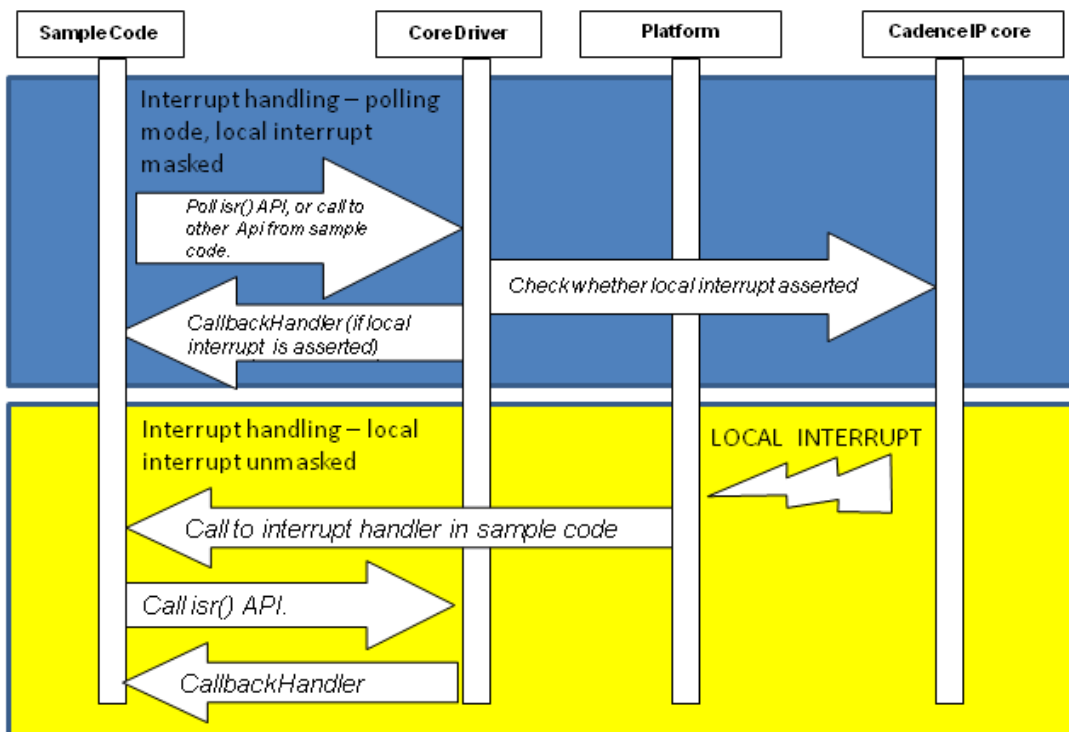
2. *Prepare interrupt handling*

The EMAC interrupts can be handled either by polling or by interrupt handling, depending on the requirements of the system. If the EMAC 'ethernet_int' interrupt signal is not connected to

an interrupt controller, or remains disabled in the controller, the status of the IP core interrupts is polled as shown in the diagram below. The driver's *isr* API should be polled regularly to determine if there are any local interrupts active, and this will call the relevant callback functions to handle any active interrupts if they have been enabled. You can register callback functions when initialising the driver, these will be enabled when the *start* API is called.

If the EMAC interrupt is to be handled by a processor interrupt, then an interrupt handler should be written and hooked up to the 'ethernet_int' output. This handler should then call the driver's interrupt handler *isr*, which will in turn call any registered callback functions. Note that currently the *isr* API function handles interrupts from all priority queues, so the separate 'ethernet_int_q*' outputs should be OR'd together with 'ethernet_int', to generate a single processor interrupt for multi-queue operation. Details of how to hook up the interrupt handler will be processor and operating system specific, and are not covered in this documentation.

Figure 1. Local Interrupts



3. Build the core-driver

The driver and a sample makefile is provided in the `core_driver` directory. You will need to customise the makefile for your build environment, e.g. compiler, any necessary libraries etc.

After this step you should have an object file `edd.o` which can be linked with your own code.

4. Use the core-driver to initialise the Cadence IP

Sample code is available in the `reference_code/tests` directory. The sample code is designed to require only minor changes before it can be used. You will however need to change some default values in `edd_test_supp.h` to match your hardware, i.e.:

```
/* IP Base addresses - re-define as appropriate
   for your hardware platform */
#define EMAC_REG_BASE_ADDRESS0    (0xFFF30000)
#define EMAC_REG_BASE_ADDRESS1    (0xFFF32000)
```

The sample code includes several test functions in `mac_test.c`, as listed in the `README.txt` file, which are called from the `main()` function in `example_main.c`. Most of the tests rely on a test environment which has two EMAC IP instances connected to each other, which is why two base addresses are defined. However, `loopbackTest` requires only a single EMAC instance, using the `EMAC_REG_BASE_ADDRESS0` address definition.

Each test calls a test setup function which creates and initialises a driver instance. This first uses *probe* to determine the size of memory which is required for private data. The memory must be provided by the code calling the driver, and may be allocated at compile time (e.g. by an array allocation) or dynamically. This private data will be passed to the driver as a parameter for each call to the API.

The sample code then prepares some example initialisation parameters, in a `CEDI_Config` struct, e.g. Tx and Rx rings and IP configuration settings, etc. You should replace these with any default settings you wish to configure: please see the user guide at `doc/emacs_driver/emacs_driver_guide.pdf` for more details on all available options.

The sample code then calls *init* to initialise the driver and the hardware. The test function continues by preparing Rx buffers, calling *start* to enable the IP interrupts (or "events"), and enabling the Tx and Rx EMAC functionality.

During the tests the sample code displays some diagnostic information using `stdio` via the '`cInst->printf`' call which is defined in `example_main.c`. If `stdio` is not available on your system, the `env_printf` operation should be defined as `NULL` or an alternative (e.g. write to memory or local storage.)

After modifying the sample code to suit your environment, you should compile for your target system and link with the driver and CPS objects prepared previously, then download and run the code on your target system. This will be hardware and operating system specific, and is not covered in this documentation. An example makefile suitable for building the driver and reference test code on a linux system is provided.